

JobHub

SUMMARY

JOBHUB — COMPLETE JOB PORTAL SYSTEM ## Master Antigravity Build Specification

PROJECT INSTRUCTION

You are an expert full-stack product engineer, UI/UX designer, frontend developer, backend developer, database architect, and QA engineer.

Build the complete JobHub — Job Portal System from scratch as a polished, responsive, production-style academic project.

This is a 1st-year BCA academic project, so the functionality must remain understandable and appropriately scoped, but the visual quality, UX, responsiveness, and overall presentation should look like a modern professional job portal.

The project must be fully functional.

Do NOT create a static mockup.

Do NOT leave unfinished placeholder sections.

Do NOT generate fake buttons that do nothing.

Every important interaction must work.

The final project must be capable of being: 1. Run locally 2. Tested locally 3. Pushed to GitHub 4. Deployed to Vercel.

CORE OBJECTIVE

JobHub connects Job Seekers, Recruiters, and Administrators.

The complete application flow should work from: Job discovery → Registration → Profile → Application → Recruiter review → Application status → Job seeker tracking.

TECHNOLOGY STACK

Frontend: Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, Lucide React icons. Backend: Next.js server-side functionality (Server Actions, Route Handlers, Server Components). Database: Supabase PostgreSQL. Authentication: Supabase Auth. Deployment: Vercel. Repository: GitHub.

DESIGN SYSTEM

Primary Color: #4F46E5 Secondary: #7C3AED Accent: #06B6D4 Success: #16A34A Warning: #F59E0B Error: #DC2626 Main Background: #F8FAFC White: #FFFFFF Main Text: #0F172A Secondary Text: #64748B Borders: #E2E8F0 Typography: Poppins

USER ROLES

Implement exactly three roles: - Job Seeker - Recruiter - Admin

DATABASE SCHEMA

Tables: - profiles - companies - jobs - applications - saved_jobs - education - notifications

Include primary keys, foreign keys, unique constraints, not-null constraints, cascading rules where appropriate, and Row Level Security policies.

QUALITY AND DELIVERY REQUIREMENTS

Run `npm install`, `npm run lint`, `npm run build`. Fix all TypeScript errors, ESLint errors, build errors, missing imports, broken routes, and invalid server/client component usage. Test responsive behavior across specified breakpoints. Test Job Seeker, Recruiter, and Admin workflows. Provide `README.md` and `.env.example`. Ensure GitHub-ready and Vercel-ready setup with no secrets committed.